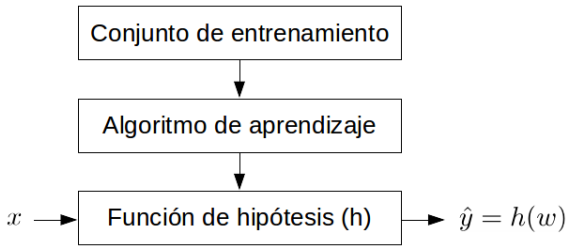
Universidad de Ingeniería y Tecnología - UTEC
Departamento de Ingeniería Mecatrónica
Curso: Tópicos de Ingeniería Mecatrónica (2020-2)

Tema 1:
Regresión Lineal

Prof. Oscar E. Ramos Ponce

El aprendizaje supervisado es una forma de aprendizaje automático donde el sistema tiene acceso a la respuesta correcta para cada instancia de los datos. En esta forma de aprendizaje existe un conjunto de entrenamiento (llamado en inglés *training set*) que se utiliza para entrenar el algoritmo de aprendizaje supervisado. Este algoritmo, una vez entrenado, determina los mejores parámetros, según alguna métrica, de una función de hipótesis h. Luego, cuando el sistema tiene como entrada una nueva instancia $\boldsymbol{x}$, se utiliza esta función de hipótesis h para obtener la predicción $\hat{y} = h(\boldsymbol{x})$. Un esquema que ilustra el comportamiento descrito se muestra en la siguiente figura.



El problema de regresión es un problema de aprendizaje supervisado en el cual se predice valores reales para la salida ($y \in \mathbb{R}$), dada una entrada $\boldsymbol{x}$ con d atributos ($\boldsymbol{x} \in \mathbb{R}^d$). En otros términos, en este problema se obtiene una función de hipótesis $h : \mathbb{R}^d \to \mathbb{R}$.

1. Notación

Se considerará que el conjunto de datos $\mathcal{D}$ utilizados para el entrenamiento contiene n instancias, también llamados *ejemplos*, y está dado por

$$\mathcal{D} = \left\{ \left(\boldsymbol{x}^{(1)}, y^{(1)} \right), \left(\boldsymbol{x}^{(2)}, y^{(2)} \right), \cdots, \left(\boldsymbol{x}^{(n)}, y^{(n)} \right) \right\}.$$

La notación que se utilizará será:

- n: número de instancias, muestras o ejemplos de entrenamiento.
- $\left(\boldsymbol{x}^{(i)}, y^{(i)} \right)$: instancia i-ésima de entrenamiento.

- $\boldsymbol{x}^{(i)} \in \mathbb{R}^d$: vector de d atributos de entrada correspondiente a la i-ésima instancia de entrenamiento, tal que

$$\boldsymbol{x}^{(i)} = \begin{bmatrix} x_1^{(i)} \\ x_2^{(i)} \\ \vdots \\ x_d^{(i)} \end{bmatrix}$$

  donde $x_j^{(i)}$ es el valor del atributo j en la instancia i-ésima de entrenamiento. A los atributos también se les denomina *características* o *features*.

- $y^{(i)} \in \mathbb{R}$: salida correspondiente a la i-ésima instancia.

De manera general, cuando no se hace referencia a una instancia específica del conjunto de datos, se tiene

- $(\boldsymbol{x}, y)$: instancia para el entrenamiento
- $\boldsymbol{x} \in \mathbb{R}^d$: vector de d atributos de entrada (*feature vector*)
- $y \in \mathbb{R}$: variable de salida (variable objetivo)

En casos donde no sea necesario hacer referencia explícita a una instancia i específica, se omitirá los superíndices i, por claridad de notación.

2. Función de Hipótesis

La *regresión lineal multivariable* utiliza una combinación lineal de cada elemento x_i, del vector de entrada $\boldsymbol{x}$, como función de hipótesis h_w, de tal modo que la predicción de la salida $\hat{y}$ queda dada por $\hat{y} = h_w(\boldsymbol{x})$. Considerando que el vector de entrada $\boldsymbol{x}$ tiene d atributos (*features*), la función de hipótesis se puede escribir como:

$$h_w(\boldsymbol{x}) = w_0 + w_1 x_1 + w_2 x_2 + w_3 x_3 + \cdots + w_d x_d \tag{1}$$

donde cada elemento w_i es un parámetro o peso que se debe determinar (se utiliza el término w por el inglés *weight*). El subíndice w en h_w hace alusión explícita a que la hipótesis depende de los parámetros w_i. En (1), al escalar w_0 se le suele denominar el sesgo, *bias* u *offset* del modelo.

De manera alternativa, y haciendo un abuso de notación, si se considera que

$$\boldsymbol{x} = \begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_d \end{bmatrix} \qquad \text{y} \qquad \boldsymbol{w} = \begin{bmatrix} w_0 \\ w_1 \\ \vdots \\ w_d \end{bmatrix},$$

con $x_0 = 1$, la función de hipótesis de la regresión lineal se puede representar como el producto punto del vector de entrada $\boldsymbol{x}$ con el vector de pesos $\boldsymbol{w}$:

$$h_w = \boldsymbol{w}^T \boldsymbol{x} = \sum_{i=0}^{d} w_i x_i. \tag{2}$$

En la práctica se puede utilizar tanto (1) como (2) de manera indistinta, siempre que se tenga cuidado de utilizar $x_0 = 1$. Por otro lado, en algunos casos resulta común usar $b = w_0$ como valor de *bias* y no incluir w_0 en $\boldsymbol{w}$, ni x_0 en $\boldsymbol{x}$, de tal modo que la función de hipótesis quede dada por $h_w(\boldsymbol{x}) = \boldsymbol{w}^T \boldsymbol{x} + b$. Sin embargo, aquí se adoptará en adelante la notación de función de hipótesis dada en (2).

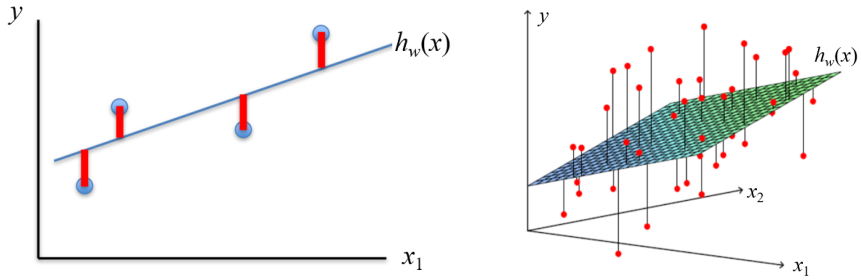


Figura 1: Residuos en la regresión lineal de una variable, para el caso monovariable (izquierda) y para el caso bivariable (derecha). Los puntos representan los valores reales y, la recta y el plano representan la función de hipótesis, y las líneas verticales representan los residuos.

Regresión Lineal Univariable. Es un caso especial de la regresión lineal multivariable en el cual $x \in \mathbb{R}$ y la hipótesis queda dada por

$$h_w(x) = w_0 + w_1 x$$

En este caso específico, cuando $x \in \mathbb{R}$, la regresión obtenida determina una línea en el espacio bidimensional que trata de ajustarse a los datos y es fácil de visualizar. En el caso general $\boldsymbol{x} \in \mathbb{R}^d$, la regresión lineal determina un hiperplano en el espacio de dimensión d, el cual, evidentemente, no puede ser visualizado.

3. Función de Costo

La función de costo para la regresión lineal, asumiendo que existen n datos o instancias de entrenamiento, está dada por

$$J(\boldsymbol{w}) = \frac{1}{2n} \sum_{i=1}^{n} \left(h_w(\boldsymbol{x}^{(i)}) - y^{(i)} \right)^2 . \tag{3}$$

Esta función se denomina función de *error cuadrático medio*, y el término $\frac{1}{2}$ aparece solo por conveniencia matemática, para que al derivar se anule con el 2 que aparecería en el numerador. Si no se utiliza el factor $\frac{1}{2n}$, la función de costo se suele denominar *suma de cuadrados residuales* (RSS: *Residual Sum of Squares*). El término $h_w(\boldsymbol{x}^{(i)}) - y^{(i)}$ representa el residuo para la instancia i; es decir, el error en el cual incurre la función de hipótesis h_w cuando utiliza la instancia $\boldsymbol{x}^{(i)}$ para predecir el valor del verdadero $y^{(i)}$. La Figura 1 muestra dichos residuos para el caso univariable y para el caso bivariable.

Dado que la función de costo $J(\boldsymbol{w})$ representa los errores acumulados en la predicción que realiza la función de hipótesis, para poder encontrar la hipótesis h_w que se ajuste mejor a los puntos de entrenamiento se debe minimizar $J(\boldsymbol{w})$. Así, el problema que se busca resolver se puede expresar como

$$\min_{w} J(\boldsymbol{w}),$$

donde J está dado por (3). La solución a este problema de minimización encontrará los parámetros $\boldsymbol{w}$ tales que h_w se encuentre lo más cercano posible a cada y de los datos de entrenamiento.

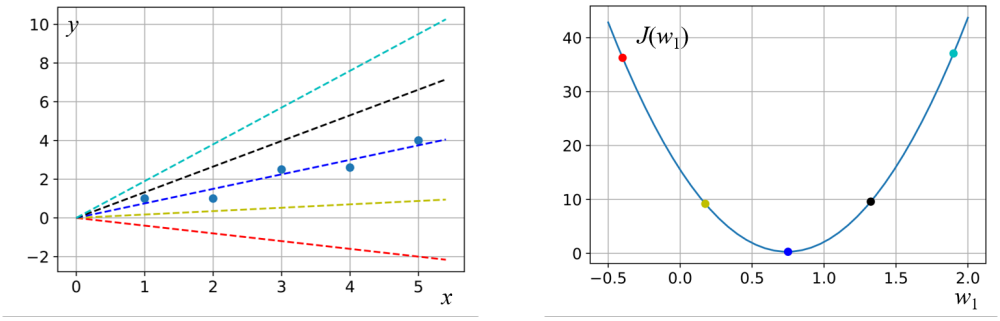


Figura 2: Puntos de entrenamiento, en azul, y líneas que representan diversas hipótesis (izquierda). Función de costo J asociada (derecha). En la función J se muestra puntos cuyos valores w_1 corresponden a las líneas de la función de hipótesis (para los mismos colores).

Ejemplo. Para el caso de la regresión lineal univariable, asumiendo que no hay sesgo o *bias* ($w_0 = 0$), la función de hipótesis es $h_w(x) = w_1 x$. Esta función se puede visualizar en el plano xy como un conjunto de líneas que pasan por el origen, donde w_1 representa la pendiente, tal como se muestra en la Figura 2 (izquierda). Reemplazando esta expresión de h_w en (3), se obtiene la función de costo $J(w_1) = \frac{1}{2n}\sum_{i=1}^{n}\left(w_1 x^{(i)} - y^{(i)}\right)^2$. Al graficar J en función de la variable independiente w_1, como se muestra en la Figura 1 (derecha), se puede notar que el costo de cada una de estas líneas representa un punto. Debido a la simplicidad de J de este ejemplo, $J(w_1)$ describe una parábola, y el valor de w_1 que lleve a la mejor hipótesis, será el valor donde la parábola tiene su mínimo.

Curvas de Nivel

En el caso univariable, si se asume que $w_0 \neq 0$, $w_1 \neq 0$, se tiene que la función de hipótesis es $h_w(x) = w_0 + w_1 x$, y la función de costo es una función de dos variables $J(w_0, w_1) = \frac{1}{2n}\sum_{i=1}^{n}\left(w_0 + w_1 x^{(i)} - y^{(i)}\right)^2$. En este caso la gráfica de J será una gráfica en el espacio tridimensional, como se muestra en la Figura 3. Para este caso particular, debido a la forma en la que se define la función de costo, la gráfica tendrá la forma de un tazón (*bowl*), y el valor más bajo corresponderá a los valores de w_0, w_1 que minimizan la función de costo y, por tanto, el error en el que incurre la función de hipótesis.

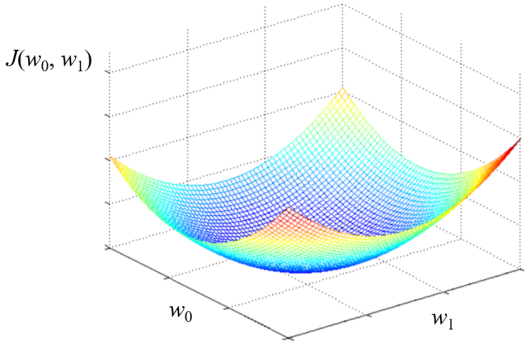


Figura 3: Función de costo J en función de los parámetros w_1 y w_2.

Debido a la complejidad de los gráficos tridimensionales, una alternativa usual consiste en graficar las curvas de nivel de la función de costo J, las cuales igualmente brindan

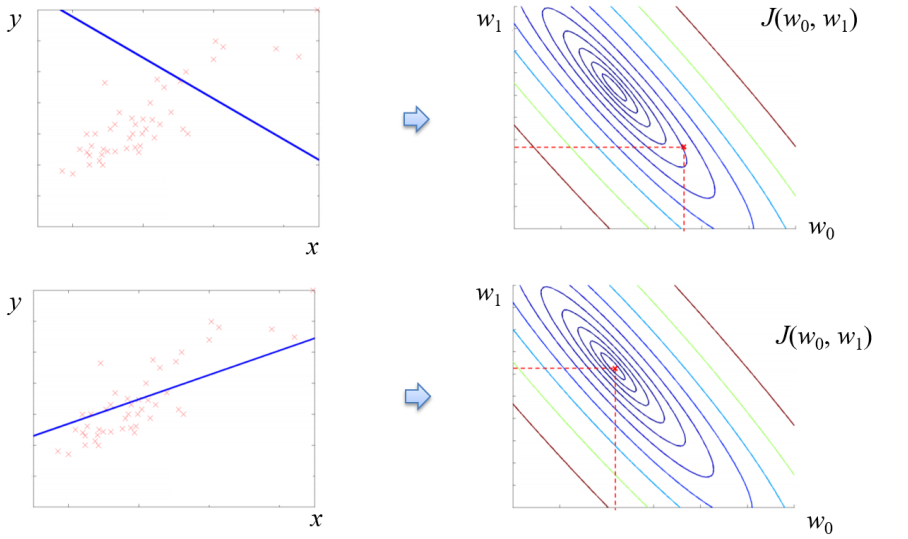


Figura 4: Funciones de hipótesis (izquierda) y puntos asociados en las curvas de nivel de la función de costo (derecha). Se puede observar en las imágenes inferiores que un buen ajuste corresponde a un punto que se encuentra en la elipse más pequeña de las curvas de nivel.

información sobre dónde se encuentra el mínimo de la función. Por ejemplo, la gráfica superior izquierda de la Figura 4 muestra un conjunto de puntos (en rojo) y una función de hipótesis ajustada (en azul). La figura superior derecha muestra las curvas de nivel de la función de costo asociada a la función de hipótesis, además de un punto cuyos parámetros w_0, w_1 son los utilizados en la función de la izquierda. En las dos figuras de la parte inferior de la Figura 4 se muestra el caso óptimo: a la izquierda la mejor curva que se ajusta a los parámetros, y a la derecha el punto de parámetros correspondiente en las curvas de nivel. Este punto óptimo se sitúa al medio de la elipse más pequeña.

4. Estimación de Parámetros: Descenso del Gradiente

La idea general para estimar parámetros adecuados consiste en comenzar con algún valor de pesos $\boldsymbol{w}$ (llamado *valor inicial*), e ir modificando estos pesos $\boldsymbol{w}$ de tal modo que con cada iteración se reduzca el costo $J(\boldsymbol{w})$, hasta alcanzar el costo mínimo.

4.1. Descenso de Gradiente Genérico

El algoritmo de descenso de gradiente (*gradient descent*) es un algoritmo iterativo que busca encontrar el peso $\boldsymbol{w}$ que minimice $J(\boldsymbol{w})$ luego de algunas iteraciones. Este algoritmo aplica la siguiente regla de actualización término a término hasta que se obtenga convergencia:

$$w_j[k+1] = w_j[k] - \alpha \frac{\partial}{\partial w_j[k]} J(\boldsymbol{w}[k]) \tag{4}$$

donde $w_j[k]$ indica el componente j-ésimo del vector de parámetros o pesos $\boldsymbol{w}$ en el instante de tiempo k, y α se denomina el factor de aprendizaje (*learning rate*). Por facilidad de notación, la expresión anterior a veces se representa como

$$w_j := w_j - \alpha \frac{\partial}{\partial w_j} J(\boldsymbol{w}),$$

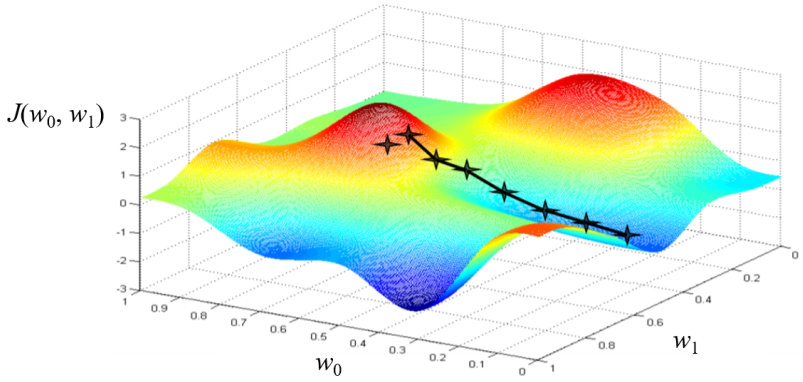


Figura 5: Función de costo $J(w_0, w_1)$ ilustrando el descenso del gradiente.

donde los términos a la derecha de la igualdad representan los valores en la iteración k y el término a la izquierda el valor en la iteración $k+1$. Equivalentemente, la notación $:=$ indica que el término de la derecha se asigna al término de la izquierda. En (4), es importante que la actualización se realice de manera simultánea para todos los j, desde 0 hasta el número de atributos d.

El criterio de término del algoritmo de descenso del gradiente se puede establecer cuando la función de costo disminuye más allá de un valor umbral prestablecido. Sin embargo, los valores de la función de costo varían según el problema que se tenga, así que determinar dicho umbral a priori no es, en general, posible. Otra alternativa consiste en repetir el algoritmo por una cantidad fija de iteraciones (numero máximo de iteraciones). Sin embargo, la alternativa más usada consiste en establecer que existe convergencia cuando la función de costo $J(\boldsymbol{w})$ decrece menos que un valor pequeño ε (por ejemplo, $\varepsilon = 10^{-3}$) en una sola iteración. Esto significa que se detiene el algoritmo en la iteración k si se cumple que

$$|J(\boldsymbol{w}[k]) - J(\boldsymbol{w}[k-1])| < \varepsilon.$$

Alternativamente, se puede observar el gráfico de $J(\boldsymbol{w})$ en función del número de iteraciones y determinar cuándo los decrementos pueden considerarse como "mínimos".

La Figura 5, donde se grafica la función de costo $J(w_0, w_1)$ en función de los parámetros w_0 y w_1, muestra un ejemplo de cómo funciona este algoritmo. Inicialmente, en $k = 0$, se utiliza un punto arbitrario, usualmente $(0, 0)$ o algún valor aleatorio, representado por $w_0[0]$ y $w_1[0]$ (en el ejemplo mostrado se inicia en un punto sobre la superficie roja). Luego se aplica la actualización de los parámetros w_0 y w_1 usando la expresión (4), obteniéndose los pares $(w_0[1], w_1[1])$, luego $(w_0[2], w_1[2])$, y así sucesivamente. Estos nuevos puntos se ilustran en la Figura 5 como puntos que aparecen a la derecha del punto inicial. Pasado un cierto número de iteraciones N se llega al valor mínimo, con coordenadas $(w_0[N], w_1[N])$, que en la figura es uno de los valles mostrados en azul. El algoritmo de descenso del gradiente depende mucho del valor inicial que se tome, ya que diferentes valores iniciales pueden llevar a diferentes mínimos locales. En la figura anterior, si se inicia en otro punto se puede terminar en otro valle azul, dado que hay más de un mínimo local. Para evitar el problema de mínimos locales, se busca funciones de costo J que sean convexas; es decir, que tengan un único mínimo global.

Una manera alternativa de visualizar el descenso del gradiente se realiza usando las curvas de nivel, como se muestra en la Figura 6. En esta figura se puede notar que se inicia con valores de pesos $(w_0[0], w_1[0])$ que definen un punto arbitrario; por ejemplo, el punto rojo más a la derecha. De allí, el algoritmo genera nuevos pesos de manera iterativa,

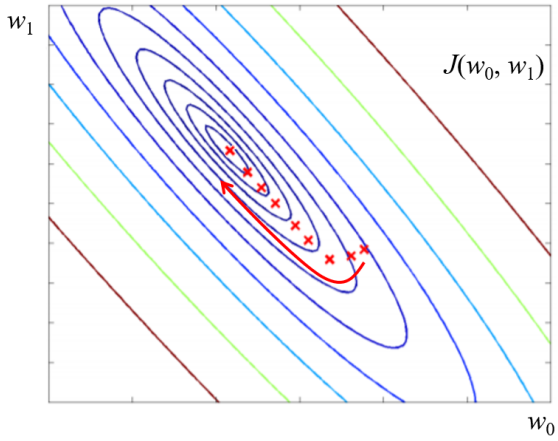


Figura 6: Curvas de nivel de la función de costo $J(w_0, w_1)$ ilustrando el descenso del gradiente.

de tal modo que se converge tal como se muestra con la flecha roja. En las curvas de nivel el objetivo consiste en llegar al centro de la elipse más pequeña, ya que dicho centro representa el mínimo local.

Al cálculo del descenso de gradiente utilizando todas las muestras del conjunto de entrenamiento se le denomina de tipo *Batch* (por lotes).

4.2. Aplicación a la Regresión Lineal

Para aplicar el algoritmo del descenso del gradiente a la función de costo asociada a la regresión lineal, se requiere obtener las derivadas parciales de la función de costo J, dada en (3), con respecto a cada uno de los pesos w_i. Estas derivadas pueden calcularse de manera analítica como

$$\begin{aligned}
\frac{\partial}{\partial w_j} J(\boldsymbol{w}) &= \frac{\partial}{\partial w_j} \frac{1}{2n} \sum_{i=1}^{n} \left(h_w(\boldsymbol{x}^{(i)}) - y^{(i)} \right)^2 \\
&= \frac{1}{2n} \frac{\partial}{\partial w_j} \sum_{i=1}^{n} \left(w_0 x_0^{(i)} + w_1 x_1^{(i)} + \cdots + w_d x_d^{(i)} - y^{(i)} \right)^2 \\
&= \frac{1}{2n} 2 \sum_{i=1}^{n} \left(h_w(\boldsymbol{x}^{(i)}) - y^{(i)} \right) \frac{\partial}{\partial w_j} \left(w_0 x_0^{(i)} + w_1 x_1^{(i)} + \cdots + w_d x_d^{(i)} - y^{(i)} \right) \\
&= \frac{1}{n} \sum_{i=1}^{n} \left(h_w(\boldsymbol{x}^{(i)}) - y^{(i)} \right) x_j^{(i)}.
\end{aligned}$$

Utilizando esta expresión de la derivada, el algoritmo de descenso de gradiente se puede expresar para la regresión lineal como

$$w_j := w_j - \alpha \frac{1}{n} \sum_{i=1}^{n} \left(h_w(\boldsymbol{x}^{(i)}) - y^{(i)} \right) x_j^{(i)}. \tag{5}$$

A esta expresión también se le conoce como la regla de mínimos cuadrados, regla *LMS* (del inglés *Least Mean Squares*) o la fórmula de aprendizaje de Widrow-Hoff. La expresión anterior se debe aplicar de forma simultánea para cada w_j. De manera vectorial se puede escribir como

$$\boldsymbol{w} := \boldsymbol{w} - \alpha \frac{1}{n} \sum_{i=1}^{n} \left(h_w(\boldsymbol{x}^{(i)}) - y^{(i)} \right) \boldsymbol{x}^{(i)}. \tag{6}$$

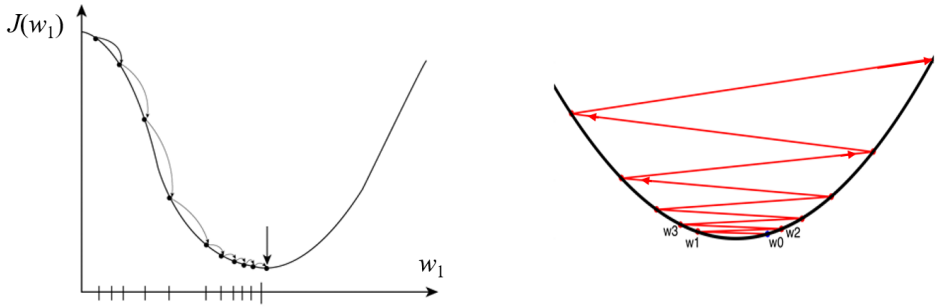


Figura 7: Ilustración de un factor de aprendizaje que converge hacia el mínimo de la función de costo (izquierda). Ilustración de un factor de aprendizaje alto que genera divergencia (derecha).

En la regresión lineal, la función de costo $J(\boldsymbol{w})$ es una función convexa, lo que significa que tiene un único mínimo global (no existen mínimos locales). De este modo, la forma de esta función tiene la forma de un *bowl*.

4.3. Factor de Aprendizaje (α)

El factor de aprendizaje es un hiperparámetro (parámetro que se fija de manera manual) cuyo valor es crítico en la convergencia del algoritmo de descenso del gradiente. Un factor adecuado permite un aprendizaje adecuado, lo cual se verifica con la reducción sucesiva de la función de costo J hasta alcanzar un mínimo, como se ilustra en la Figura 7 (izquierda). Los valores extremos del factor de aprendizaje (α) presentan los siguientes casos:

- Si α es muy pequeño, el descenso del gradiente es lento. Esto significa que la función de costo $J(\boldsymbol{w})$ se hace más pequeña en cada iteración pero de manera muy lenta; es decir, existe convergencia, pero es muy lenta.
- Si α es muy grande, el descenso del gradiente puede hacer un *overshoot* del mínimo y puede divergir. En este caso, la función de costo $J(\boldsymbol{w})$ puede hacerse pequeña en algunas iteraciones y luego incrementarse hacia el infinito, típicamente dando saltos entre ambos extremos de la función de costo. Este comportamiento se ilustra en la Figura 7 (derecha).

Para asegurarse de tener un aprendizaje adecuado, se debe verificar que la función de costo $J(\boldsymbol{w})$ disminuya luego de cada iteración. En caso que disminuyese muy lento, se puede incrementar el factor de aprendizaje, dado que podría ser muy pequeño. Por el contrario, si la función de costo se hace cada vez más grande (diverje), se debe reducir el valor del factor de aprendizaje.

Factor de Aprendizaje adecuado. El número de iteraciones necesario para la convergencia varía según la aplicación que se esté trabajando. En general, mientras más pequeño sea α más lenta será la convergencia, así que se busca un valor de α lo más grande posible sin que lleve a divergencia. Para escoger un valor adecuado del factor de aprendizaje se puede comenzar usando un valor pequeño y luego este valor se va incrementando por un factor determinado hasta llegar a un valor donde el descenso del gradiente diverja. Por ejemplo, se puede comenzar con $\alpha = 0.001$, luego con $\alpha = 0.01$, y así hasta llegar a un valor que haga que la función diverja. De este modo, el valor anterior a la divergencia (cuando aún había convergencia) se puede usar, en adelante, como factor de aprendizaje. Incluso se puede graficar $J(\boldsymbol{w})$ para cada valor de α con el fin de ver gráficamente qué valor permite una convergencia más rápida.

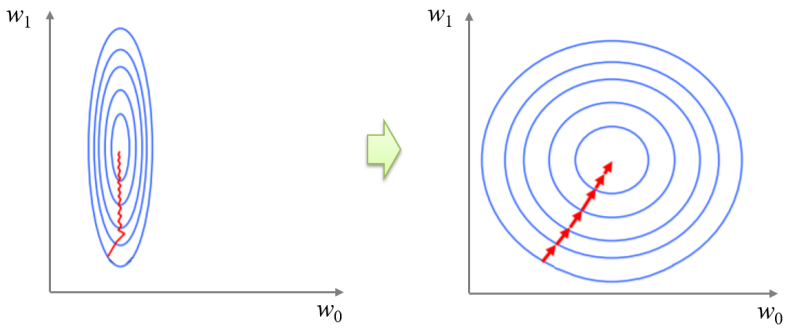


Figura 8: Efecto de la escala: con escalas diferentes el descenso del gradiente demora más y oscila más (izquierda); con escalas similares el descenso del gradiente es más uniforme (derecha).

4.4. Modificación de la escala de los Atributos

El objetivo de modificar la escala de los atributos de $\boldsymbol{x}$ (en inglés llamado *feature scaling*) es el obtener atributos con escalas similares. Esto es importante ya que cuando los atributos tienen escalas muy diferentes (por ejemplo, $x_1 \in [0, 1]$ pero $x_2 \in [-1000, 1000]$), el descenso del gradiente puede converger muy lentamente o hasta oscilar. Este efecto se ilustra en la Figura 8.

Existen dos formas principales para modificar la escala:

- *Modificación Simple.* Para cada atributo, divide los valores entre valor más grande de dicho atributo, con el fin de obtener valores en el rango -1 a 1 o de 0 a 1, aproximadamente. Se asume que los valores están aproximadamente centrados en cero.

- *Normalización de la media.* Hace que cada atributo esté centrado en su media aritmética utilizando:

$$x_i = \frac{x_i - \mu_i}{\sigma_i}$$

  donde μ_i es la media aritmética del atributo i, y σ_i es su desviación estándar. A veces se utiliza el rango (valor máximo menos el valor mínimo) en lugar de la desviación estándar.

Estas formas de escalar no son rigurosas: los rangos que se busca obtener son aproximados pero se espera que sean cercanos. Es importante tener en cuenta que no se debe normalizar el valor de x_0 ya que se asume que siempre es 1.

Ejemplo. Si se tiene dos atributos tales que $x_i \in \{0, 2000\}$ y $x_2 \in \{1, 5\}$, el descenso del gradiente puede oscilar y tomar un tiempo muy largo en alcanzar convergencia. Esto se puede verificar graficando las curvas de nivel de la función de costo. En este caso, una alternativa es escalar para tener los nuevos atributos $\tilde{x}_1$, $\tilde{x}_2$ de tal modo que $\tilde{x}_1 \in \frac{x_1}{2000}$, y $\tilde{x}_2 = \frac{x_2}{5}$. Así se obtendrá a ambos atributos aproximadamente con valores entre 0 y 1.

5. Solución Analítica: Ecuación Normal

Anteriormente se ha utilizado el descenso del gradiente para obtener el mínimo de la función de costo. Para obtener este mínimo de forma analítica se puede utilizar el álgebra

lineal de la siguiente forma. Defínase el vector $\boldsymbol{h}_w(X)$ que contiene las hipótesis de todas las muestras de tal modo que:

$$\boldsymbol{h}_w(X) = \begin{bmatrix} h_w(\boldsymbol{x}^{(1)}) \\ h_w(\boldsymbol{x}^{(2)}) \\ \vdots \\ h_w(\boldsymbol{x}^{(n)}) \end{bmatrix} = \begin{bmatrix} \boldsymbol{w}^T\boldsymbol{x}^{(1)} \\ \boldsymbol{w}^T\boldsymbol{x}^{(2)} \\ \vdots \\ \boldsymbol{w}^T\boldsymbol{x}^{(n)} \end{bmatrix} = \begin{bmatrix} (\boldsymbol{x}^{(1)})^T\boldsymbol{w} \\ (\boldsymbol{x}^{(2)})^T\boldsymbol{w} \\ \vdots \\ (\boldsymbol{x}^{(n)})^T\boldsymbol{w} \end{bmatrix} = \begin{bmatrix} (\boldsymbol{x}^{(1)})^T \\ (\boldsymbol{x}^{(2)})^T \\ \vdots \\ (\boldsymbol{x}^{(n)})^T \end{bmatrix} \boldsymbol{w}.$$

Además, se definirá la matriz X y el vector $\boldsymbol{y}$ como

$$X = \begin{bmatrix} (\boldsymbol{x}^{(1)})^T \\ (\boldsymbol{x}^{(2)})^T \\ \vdots \\ (\boldsymbol{x}^{(n)})^T \end{bmatrix}, \qquad \boldsymbol{y} = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(n)} \end{bmatrix}$$

de tal modo que $\boldsymbol{h}_w(X) = X\boldsymbol{w}$. Usando esta notación, la función de costo se puede escribir como

$$J(\boldsymbol{w}) = \frac{1}{2n}\sum_{i=1}^{n}\left(h_w(\boldsymbol{x}^{(i)}) - y^{(i)}\right)^2 = \frac{1}{2n}\sum_{i=1}^{n}\left(\boldsymbol{w}^T\boldsymbol{x}^{(i)} - y^{(i)}\right)^2 = \frac{1}{2n}\|X\boldsymbol{w} - \boldsymbol{y}\|^2,$$

donde se ha eliminado la sumatoria. Eliminando la norma L-2 de la función de costo se tiene

$$\begin{aligned} J(\boldsymbol{w}) &= \frac{1}{2n}\|X\boldsymbol{w} - \boldsymbol{y}\|^2 \\ &= \frac{1}{2n}(X\boldsymbol{w} - \boldsymbol{y})^T(X\boldsymbol{w} - \boldsymbol{y}) \\ &= \frac{1}{2n}(\boldsymbol{w}^TX^TX\boldsymbol{w} - 2\boldsymbol{y}^TX\boldsymbol{w} + \boldsymbol{y}^T\boldsymbol{y}). \end{aligned}$$

El mínimo de esta función de costo se puede obtener tomando la derivada de $J(\boldsymbol{w})$ e igualando este término a cero. Así se tiene:

$$\frac{d}{d\boldsymbol{w}}J(\boldsymbol{w}) = \frac{1}{2n}(2\boldsymbol{w}^TX^TX - 2\boldsymbol{y}^TX) = 0.$$

Resolviendo esta igualdad se obtiene $\boldsymbol{w}^TX^TX = \boldsymbol{y}^TX$. Tomando la transpuesta de esta igualdad se tiene $X^TX\boldsymbol{w} = X^T\boldsymbol{y}$, de donde se puede despejar el valor de $\boldsymbol{w}$ como

$$\boldsymbol{w} = (X^TX)^{-1}X^T\boldsymbol{y}. \tag{7}$$

A esta expresión se denomina la *ecuación normal*. A partir de la definición de X, se sabe que $X \in \mathbb{R}^{n\times d}$, y por tanto $X^TX \in \mathbb{R}^{d\times d}$. Debido a que en la práctica $n > d$ (se tiene muchas más instancias que número de atributos), la matriz X es de rango completo por columnas, y la matriz X^TX es a su vez de rango completo, encontrándose bien definida su inversa $(X^TX)^{-1}$.

Para una matriz $X \in \mathbb{R}^{n\times d}$ de rango completo por columnas, la pseudo-inversa a la izquierda de Moore Penrose $X^{\#}$ se define como

$$X^{\#} = (X^TX)^{-1}X^T.$$

Usualmente, y por brevedad de terminología, se hace referencia a esta matriz simplemente como la *pseudo-inversa*. Así, (7) se puede expresar en función de esta pseudo-inversa como

$$\boldsymbol{w} = X^{\#}\boldsymbol{y}.$$

Comparación: Descenso Gradiente - Ecuación Normal

Las características del cálculo de $\boldsymbol{w}$ usando el descenso del gradiente son:

- Se necesita escoger el valor del factor de aprendizaje α
- Necesita varias iteraciones
- La principal ventaja es que trabaja bastante bien cuando se tiene bastantes instancias (cuando n es grande)

Las características del cálculo de $\boldsymbol{w}$ usando la ecuación normal son:

- No se necesita escoger α
- No hay ninguna iteración (es un método analítico)
- Necesita calcular $(X^T X)^{-1}$, lo cual es una desventaja cuando se tiene un número muy grande de instancias (el cálculo de la inversa es $\mathcal{O}(n^3)$, lo cual hace que no sea adecuado si hay muchas instancias).
- Es lento cuando hay muchas instancias (n grande).

En resumen, es conveniente utilizar la ecuación normal cuando hay pocas instancias, pero cuando hay muchas instancias es más conveniente (y a veces la única alternativa) utilizar el descenso del gradiente.

6. Extensión: Bases no Lineales

De manera general se puede definir una función de hipótesis como una combinación lineal de bases no lineales $\phi_i(\boldsymbol{x})$ como

$$h_w(\boldsymbol{x}) = \boldsymbol{w}^T \boldsymbol{\phi}(\boldsymbol{x}) = \sum_{i=0}^{n} w_i \phi_i(\boldsymbol{x})$$

donde

$$\phi(\boldsymbol{x}) = \begin{bmatrix} \phi_0(\boldsymbol{x}) \\ \phi_1(\boldsymbol{x}) \\ \vdots \\ \phi_n(\boldsymbol{x}) \end{bmatrix}$$

es el vector de bases no lineales y $\phi_i(\boldsymbol{x})$ es una base no lineal. Usualmente, al igual que para la regresión lineal, $\phi_0(\boldsymbol{x}) = 1$, de tal modo que w_0 se comporta como un *bias*. Las bases no lineales pueden ser de varios tipos:

- *Bases lineales*: $\phi_i(\boldsymbol{x}) = x_i$.
- *Bases polinomiales*: $\phi_i(\boldsymbol{x}) = x^i$.
- *Bases Gaussianas*: $\phi_i(\boldsymbol{x}) = e^{-\frac{1}{2}\left(\frac{x-\mu_i}{s}\right)^2}$, donde μ_i, s son parámetros que afectan la posición y ancho de la base
- *Bases sigmoideas*: $\phi_i(\boldsymbol{x}) = \sigma\left(\frac{x-\mu_i}{s}\right)$, donde la función sigmoidea es $\sigma(z) = \frac{1}{1+e^{-z}}$, y los parámetros μ_i, s controlan la posición y la pendiente.

- *Otras bases:* se puede utilizar otras funciones trascendentales que estrictamente no son bases pero que sí permiten capturar la no linealidad del modelo. Por ejemplo, se puede usar funciones logarítmicas, trigonométricas, etc.

Una vez que se ha reemplazado los datos con las salidas de las bases no lineales, se puede encontrar los mejores parámetros $\boldsymbol{w}$ de la misma manera como se realizó para la regresión lineal.

De todas estas bases, la más utilizada es la base polinomial. Con esta base, la función de hipótesis se puede escribir como

$$h_w(\boldsymbol{x}) = w_0 + w_1 x + w_2 x^2 + \cdots + w_n x^n = \sum_{i=0}^{n} w_i x^i.$$

Al uso de esta base polinomial se le suele denominar *regresión polinomial*.

Ejemplo 1. Considérese el modelo

$$h_w(\boldsymbol{x}) = w_0 + w_1 x + w_2 x^2 + w_3 x^3.$$

Si se toma los atributos como $x_1 = x, x_2 = x^2, x_3 = x^3$, este modelo se puede reformular como

$$h_w(\boldsymbol{x}) = w_0 + w_1 x_1 + w_2 x_2 + w_3 x_3,$$

el cual es un modelo de regresión lineal y se puede utilizar los métodos anteriormente vistos.

Ejemplo 2. Considérese el modelo

$$h_w(\boldsymbol{x}) = w_0 + w_1 \sqrt{x} + w_2 x^5.$$

Si se hace el cambio de variable $x_1 = \sqrt{x}, x_2 = x^5$, este modelo se convierte en

$$h_w(\boldsymbol{x}) = w_0 + w_1 x_1 + w_2 x_2,$$

el cual nuevamente es un modelo de regresión lineal.

7. Otras formas de Optimización

Existen otros algoritmos más sofisticados para realizar la optimización. Algunos usados frecuentemente son

- Método del gradiente conjugado
- Algoritmo BFGS (Broyden-Fletcher-Goldfarb-Shanno)
- Algoritmo L-BFGS (BFGS de memoria limitada)

La ventaja de estos algoritmos es que no es necesario determinar un valor de α, y normalmente son más rápidos que el descenso del gradiente. La principal desventaja es que son más complejos a nivel teórico. Sin embargo, incluso estos métodos siempre necesitan el valor de la función de costo $J(\boldsymbol{w})$ y de su derivada.

Tipos de Descenso de Gradiente (GD)

- *Descenso de Gradiente Batch (BGD).* Es el descrito anteriormente en (5). La característica principal es que hace uso de todas las n instancias del conjunto de entrenamiento en cada paso de iteración (notar que en (5), la sumatoria se realiza desde 1 hasta n). En otros términos, se calcula el error para cada instancia en el conjunto de entrenamiento y solo se realiza la actualización de los parámetros luego de que todas las instancias de entrenamiento han sido evaluadas. Si existen muchas instancias (n es muy grande), es computacionalmente costoso.
- *Descenso de Gradiente Mini-Batch.* Consiste en dividir el conjunto de entrenamiento en pequeños subconjuntos denominados *mini-batches*, cada uno con n_b instancias. En este caso, la sumatoria en (5) se realiza sobre el número de elementos n_d en el subconjunto (*mini-batch*), y se va alternando de *mini-batch* en cada iteración. Esta implementación busca acelerar el proceso de entrenamiento utilizando menos datos en cada iteración (n_b en comparación con n).
- *Descenso de Gradiente Estocástico (SGD).* Calcula el error del descenso del gradiente utilizando solamente una instancia en cada iteración. Es decir, en cada iteración, la sumatoria de (5) en realidad no es una sumatoria ya que utiliza solo una instancia i-ésima escogida de manera estocástica (aleatoria). La ventaja es que es rápido, puede tener progresos de inmediato sin esperar a tener todos los datos, y puede actualizarse rápidamente con cada nuevo dato, por lo que también se denomina descenso de gradiente *online*. A pesar de que se aproxima rápidamente al mínimo, la función de costo podría no llegar a converger sino que podría oscilar alrededor del mínimo (lo cual en la práctica podría ser una buena aproximación).

8. Implementación de Regresión Lineal en Python

La implementación en Python de la regresión lineal se puede realizar de varias maneras pero en la medida de lo posible se trata de utilizar vectorización para que los cálculos sean más rápidos. Esta sección muestra una manera de vectorizar el problema para implementarlo, haciendo uso de un concepto llamado *broadcasting*, que resulta útil en Python.

8.1. Función de Hipótesis

La función de hipótesis dada por (2) es aplicable a una sola instancia del conjunto de entrenamiento. Para realizar la implementación en Python, debido al *broadcasting* que hace Python, y debido a que esta estructura también se utilizará más adelante para otros métodos, se separará el término de *bias*, w_0 del vector $\boldsymbol{w}$. De esta manera, la función de hipótesis se reescribirá como

$$h_w(\boldsymbol{w}) = \boldsymbol{w}^T \boldsymbol{x} + w_0$$

donde

$$\boldsymbol{w} = \begin{bmatrix} w_1 \\ \vdots \\ w_d \end{bmatrix}, \qquad \boldsymbol{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_d \end{bmatrix},$$

tal que $\boldsymbol{w} \in \mathbb{R}^d$, $\boldsymbol{x} \in \mathbb{R}^d$, $w_0 \in \mathbb{R}$. Para aplicar la función de hipótesis a todos los datos de entrada, se puede definir una matriz $X \in \mathbb{R}^{d \times n}$ cuyas columnas sean cada una de las n instancias de entrada $\boldsymbol{x}^{(i)}$ como se muestra a continuación

$$X = \begin{bmatrix} \boldsymbol{x}^{(1)} & \boldsymbol{x}^{(2)} & \cdots & \boldsymbol{x}^{(n)} \end{bmatrix}.$$

Luego, la función de hipótesis se aplica de una sola vez a todos los datos de entrenamiento, representados por X, como

$$\boldsymbol{h}_w(X) = \boldsymbol{w}^T X + \boldsymbol{w}_0, \tag{8}$$

donde $\boldsymbol{h}_w(X) \in \mathbb{R}^{1\times n}$ es un vector fila, y $\boldsymbol{w}_0 \in \mathbb{R}^{1\times n}$ es también un vector fila cuyos elementos son w_0. De manera explícita, los componentes de $\boldsymbol{h}_w(X)$ son

$$\begin{aligned}\boldsymbol{h}_w(X) &= \boldsymbol{w}^T \begin{bmatrix}\boldsymbol{x}^{(1)} & \boldsymbol{x}^{(2)} & \cdots & \boldsymbol{x}^{(n)}\end{bmatrix} + \begin{bmatrix}w_0 & w_0 & \cdots & w_0\end{bmatrix} \\ &= \begin{bmatrix}\boldsymbol{w}^T\boldsymbol{x}^{(1)} + w_0 & \boldsymbol{w}^T\boldsymbol{x}^{(2)} + w_0 & \cdots & \boldsymbol{w}^T\boldsymbol{x}^{(n)} + w_0\end{bmatrix} \\ &= \begin{bmatrix}h_w(\boldsymbol{x}^{(1)}) & h_w(\boldsymbol{x}^{(2)}) & \cdots & h_w(\boldsymbol{x}^{(n)})\end{bmatrix},\end{aligned}$$

lo cual muestra que efectivamente se está aplicando la función de hipótesis a cada una de las instancias $\boldsymbol{x}^{(i)}$. Un detalle de implementación es que en (8) no es necesario crear un vector $\boldsymbol{w}_0$ sino se puede utilizar directamente el valor de w_0. A pesar de que esto es matemáticamente incorrecto, al ser implementado en Python se realiza internamente un *broadcasting* que aplica dicho escalar a cada elemento del vector. Así, en Python solo será necesario implementar: $\boldsymbol{h}_w(X) = \boldsymbol{w}^T X + w_0$, con $w_0 \in \mathbb{R}$.

8.2. Función de Costo

Con el fin de vectorizar la función de costo, y por facilidad de notación, se definirá el vector $\boldsymbol{e} \in \mathbb{R}^{1\times n}$ de la siguiente manera:

$$\boldsymbol{e} = \boldsymbol{h}_w(X) - \boldsymbol{y}$$

donde $\boldsymbol{y} \in \mathbb{R}^{1\times n}$ es el vector columna que contiene todas las salidas de los datos como

$$\boldsymbol{y} = \begin{bmatrix}y^{(1)} & y^{(2)} & \cdots & y^{(n)}\end{bmatrix}.$$

De manera explícita, cada uno de los componentes de $\boldsymbol{e}$ es

$$\begin{aligned}\boldsymbol{e} &= \begin{bmatrix}h_w(\boldsymbol{x}^{(1)}) - y^{(1)} & h_w(\boldsymbol{x}^{(2)}) - y^{(2)} & \cdots & h_w(\boldsymbol{x}^{(n)}) - y^{(n)}\end{bmatrix} \\ &= \begin{bmatrix}e^{(1)} & e^{(2)} & \cdots & e^{(n)}\end{bmatrix}\end{aligned}$$

donde $e^{(i)} = h_w(\boldsymbol{x}^{(i)}) - y^{(i)}$. Utilizando este vector, la función de costo dada en (3) se puede reescribir como

$$\begin{aligned}J(\boldsymbol{w}) &= \frac{1}{2n}\sum_{i=1}^{n}(e^{(i)})^2 \\ &= \frac{1}{2n}\left((e^{(1)})^2 + (e^{(2)})^2 + \cdots + (e^{(n)})^2\right) \\ &= \frac{1}{2n}\boldsymbol{e}\boldsymbol{e}^T.\end{aligned}$$

Esta última expresión se puede implementar directamente en Python.

De manera alternativa, como se muestra en la expresión anterior, también se puede elevar cada uno de los términos del vector $\boldsymbol{e}$ al cuadrado (usando `np.square`) y luego sumar los elementos resultantes del vector, usando `np.sum`. Así, la función de costo también se podría calcular como `1/(2*n)*np.sum(np.square(e))`.

8.3. Descenso del Gradiente

El descenso del gradiente dado por (6) se puede separar en una parte correspondiente a w_0 y en otra parte correspondiente a $\boldsymbol{w} = (w_1, w_2, \cdots, w_d)$.

Para w_0, resulta más fácil utilizar la expresión (5) con $j = 0$. Debido a que en dicho caso se asumía $x_0^{(i)} = 1$, el descenso del gradiente se puede simplificar a

$$\begin{aligned} w_0 &:= w_0 - \alpha \frac{1}{n} \sum_{i=1}^{n} \left(h_w(\boldsymbol{x}^{(i)}) - y^{(i)} \right) \\ &:= w_0 - \alpha \frac{1}{n} \sum_{i=1}^{n} e^{(i)}. \end{aligned}$$

En este último término se hace necesario sumar cada elemento $e^{(i)}$ del vector $\boldsymbol{e}$, lo cual se puede realizar en Python utilizando `np.sum(e)`. Así, esta implementación queda dada como `w0-a/n*np.sum(e)`, donde a es el valor de α.

Para w (desde w_1 hasta w_d), si se utiliza la definición dada anteriormente del vector fila $\boldsymbol{e}$, la regla del descenso del gradiente de (6) se puede escribir como

$$\begin{aligned} \boldsymbol{w} &:= \boldsymbol{w} - \alpha \frac{1}{n} \sum_{i=1}^{n} e^{(i)} \boldsymbol{x}^{(i)} \\ &:= \boldsymbol{w} - \alpha \frac{1}{n} \left(e^{(1)} \boldsymbol{x}^{(1)} + e^{(2)} \boldsymbol{x}^{(2)} + \cdots + e^{(n)} \boldsymbol{x}^{(n)} \right) \\ &:= \boldsymbol{w} - \alpha \frac{1}{n} \begin{bmatrix} \boldsymbol{x}^{(1)} & \boldsymbol{x}^{(2)} & \cdots & \boldsymbol{x}^{(n)} \end{bmatrix} \begin{bmatrix} e^{(1)} \\ e^{(2)} \\ \vdots \\ e^{(n)} \end{bmatrix} \\ &:= \boldsymbol{w} - \alpha \frac{1}{n} X \boldsymbol{e}^T. \end{aligned}$$

De esta manera, la actualización del vector de parámetros queda vectorizada.